

LAPORAN PRAKTIKUM

MATA KULIAH PEMROGRAMAN WEB

Modul 12: Membuat Aplikasi React Blog Post Menggunakan API



Oleh:

Davin Aileenskie Sofiandy

707012500135

D4SIKC 49-04

PROGRAM STUDI S.Ter SISTEM INFORMASI KOTA CERDAS

FAKULTAS ILMU TERAPAN

UNIVERSITAS TELKOM

BANDUNG

2026

2. Tujuan Praktikum

- Memahami penerapan React Router untuk navigasi multi halaman pada aplikasi React.
- Menerapkan autentikasi sederhana berbasis token untuk proteksi halaman.
- Mengintegrasikan frontend React dengan REST API menggunakan Axios.
- Membuat fitur menampilkan daftar post dan detail post dari data API.
- Menerapkan antarmuka responsif menggunakan Tailwind CSS.

3. Alat dan Bahan

- Laptop/PC dengan sistem operasi Windows.
- Node.js dan npm.
- Visual Studio Code.
- React + Vite.
- Tailwind CSS, Axios, dan React Router DOM.
- XAMPP/Apache untuk menjalankan API backend hasil Modul 9.
- Web browser (Google Chrome/Microsoft Edge).

4. Langkah-Langkah Praktikum

1. Menjalankan kembali project REST API dari Modul 9 pada folder htdocs dan memastikan endpoint API dapat diakses.
2. Membuat project React baru menggunakan Vite dan memilih varian JavaScript.
3. Menginstal dependency utama: axios dan react-router-dom.
4. Menginstal serta mengonfigurasi Tailwind CSS pada project React.
5. Menyusun struktur folder aplikasi: components, pages, context, dan services.
6. Membuat konfigurasi routing di App.jsx untuk halaman Login, PostList, dan PostDetail.
7. Membuat komponen ProtectedRoute agar halaman posts hanya bisa diakses setelah login.
8. Membuat AuthContext untuk manajemen state user, token, login, logout, dan status loading.
9. Membuat service API (api.js) untuk endpoint autentikasi dan post, termasuk pengiriman header token.

10. Membangun halaman Login dengan form email/password dan validasi respons login API.

11. Membangun halaman PostList untuk mengambil data post dari API dan menampilkannya dalam bentuk kartu.

12. Membangun halaman PostDetail untuk menampilkan detail konten berdasarkan id post.

13. Menjalankan aplikasi dan menguji alur login, daftar post, serta detail post.

14. Melakukan build produksi dengan npm run build dan menghasilkan folder dist untuk deployment.

5. Hasil dan Dokumentasi Program

Hasil praktikum adalah aplikasi frontend blog post berbasis React yang terhubung dengan REST API. Aplikasi memiliki fitur autentikasi, proteksi halaman, daftar post, dan detail post.

a) Konfigurasi routing utama pada App.jsx

```
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';

import { AuthProvider } from '../context/AuthContext';

import ProtectedRoute from '../components/ProtectedRoute';

import Login from '../pages/Login';

import PostList from '../pages/PostList';

import PostDetail from '../pages/PostDetail';

function App() {

  return (

    <AuthProvider>

      <Router>

        <Routes>

          <Route path="/login" element={<Login />} />

          <Route path="/posts" element={<ProtectedRoute><PostList /></ProtectedRoute>} />

        </Routes>

      </Router>

    </AuthProvider>

  );

}
```

```

        <Route path="/posts/:id" element={<ProtectedRoute><PostDetail
/></ProtectedRoute>} />

        <Route path="/" element={<Navigate to="/posts" replace />} />

    </Routes>

</Router>

</AuthProvider>

);
}

```

b) Proteksi route pada components/ProtectedRoute.jsx

```

import { Navigate } from 'react-router-dom';

import { useAuth } from '../context/AuthContext';

const ProtectedRoute = ({ children }) => {

    const { isAuthenticated, loading } = useAuth();

    if (loading) return <p>Loading...</p>;

    if (!isAuthenticated) return <Navigate to="/login" replace />;

    return children;

};

```

c) Manajemen autentikasi pada context/AuthContext.jsx

```

const login = async (email, password) => {

    const response = await authAPI.login(email, password);

    const accessToken = response.data?.access_token || response.data?.token;

    const userData = response.data?.user || response.data?.data;

    localStorage.setItem('token', accessToken);

    localStorage.setItem('user', JSON.stringify(userData));

    setToken(accessToken);

    setUser(userData);

    return { success: true };
}

```

```
};
```

d) Konfigurasi Axios dan endpoint API pada services/api.js

```
import axios from 'axios';
```

```
const API_URL = '/ci3_project/api';
```

```
const api = axios.create({ baseURL: API_URL });
```

```
api.interceptors.request.use((config) => {
```

```
  const token = localStorage.getItem('token');
```

```
  if (token) config.headers.Authorization = `Bearer ${token}`;
```

```
  return config;
```

```
});
```

```
export const postsAPI = {
```

```
  getAll: (page = 1, perPage = 10) =>
```

```
  api.get(`/post?page=${page}&per_page=${perPage}`),
```

```
  getById: (id) => api.get(`/post/${id}`),
```

```
};
```

e) Pengambilan data daftar post pada pages/PostList.jsx

```
const [posts, setPosts] = useState([]);
```

```
const [loading, setLoading] = useState(true);
```

```
const fetchPosts = async () => {
```

```
  setLoading(true);
```

```
  const response = await postsAPI.getAll();
```

```
  setPosts(response.data);
```

```
  setLoading(false);
```

```
};
```

```
useEffect(() => {
```

```
    fetchPosts();  
  }, []);
```

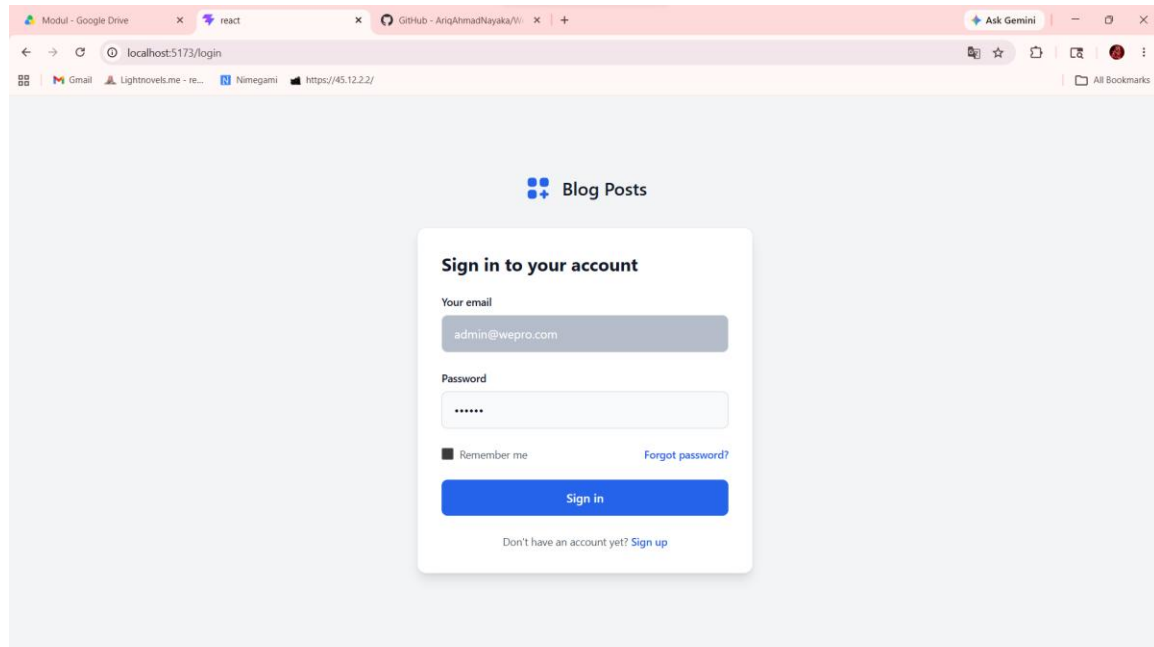
f) Pengambilan detail post berdasarkan id pada pages/PostDetail.jsx

```
const { id } = useParams();  
  
const [post, setPost] = useState(null);  
  
const fetchPost = async () => {  
  const response = await postsAPI.getById(id);  
  setPost(response.data);  
};  
  
useEffect(() => {  
  fetchPost();  
}, [id]);
```

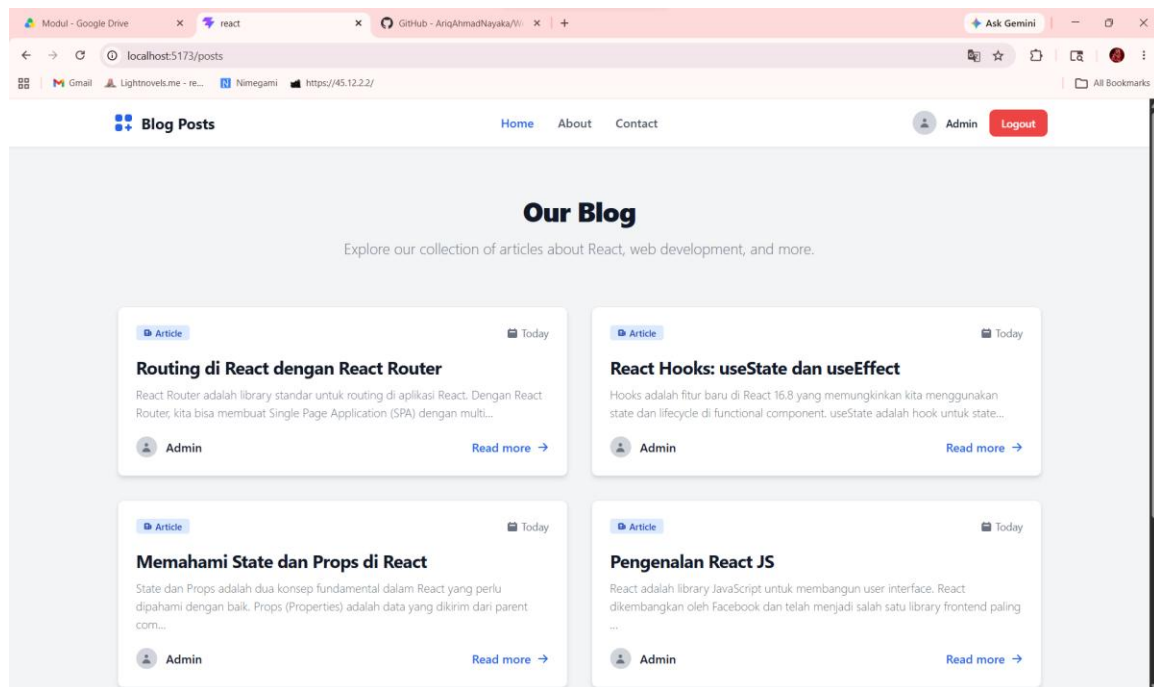
Skenario uji yang dilakukan:

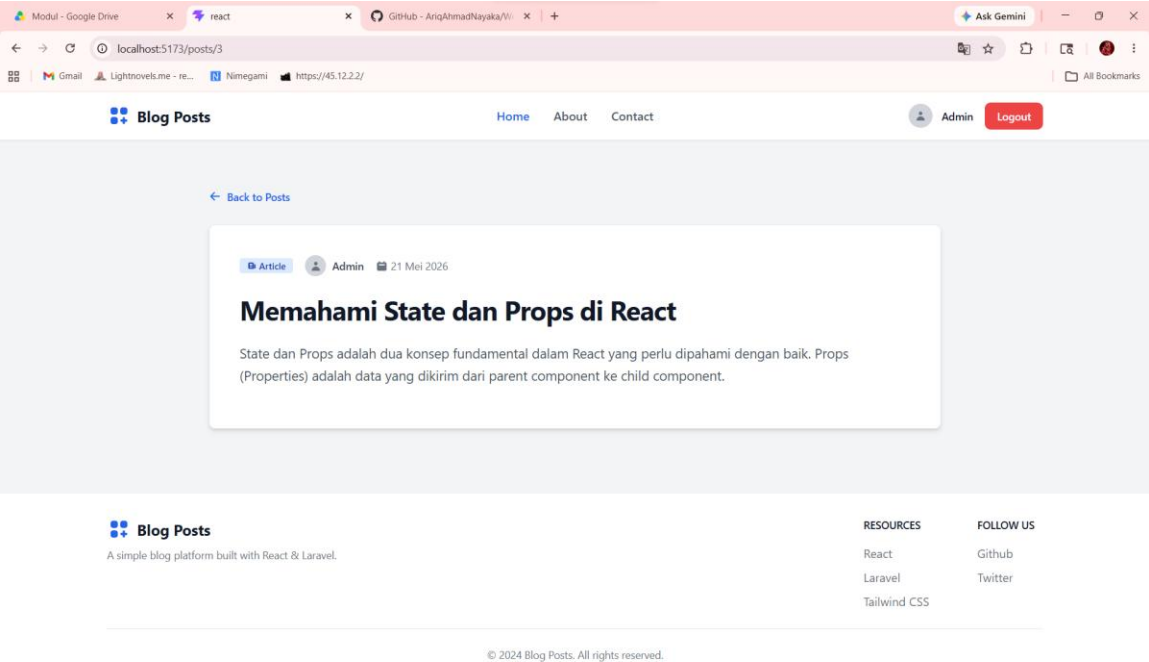
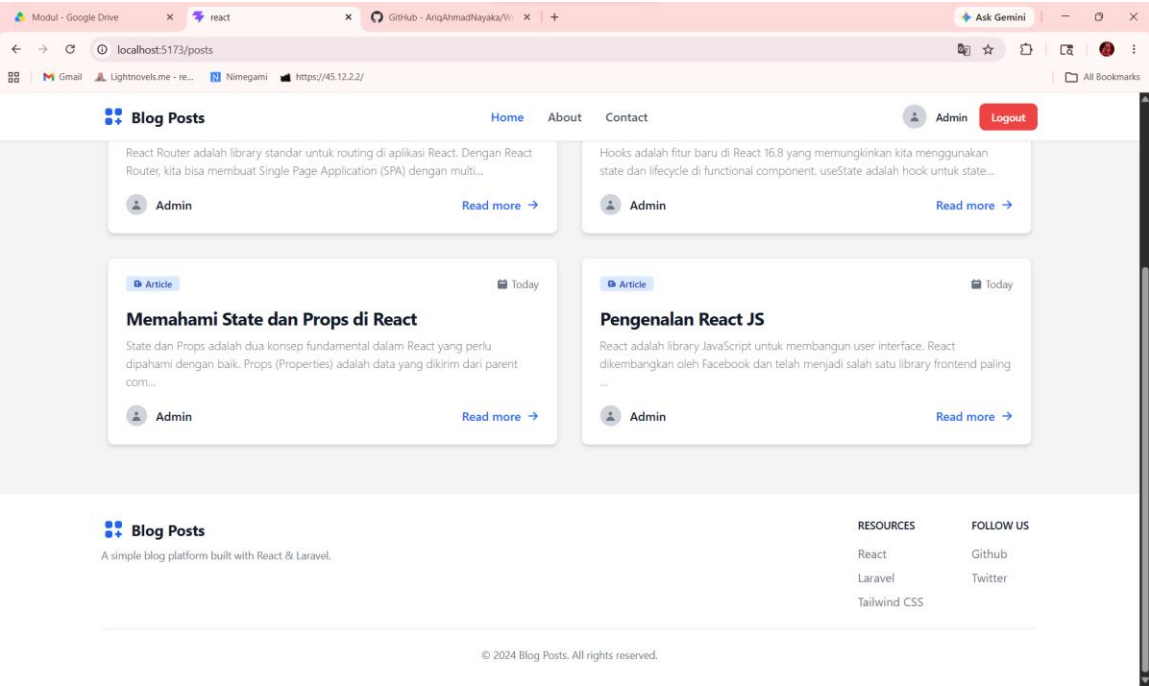
- Akses /posts tanpa login diarahkan ke /login.
- Login valid berhasil masuk ke halaman daftar post.
- Data post berhasil ditampilkan dari API pada halaman PostList.
- Klik salah satu post berhasil membuka halaman detail berdasarkan id post.
- Jika API gagal diakses, sistem menampilkan pesan error dan tombol coba lagi.

Output



© 2024 Blog Posts. All rights reserved.





6. Analisis dan Pembahasan

Pada praktikum ini, React berperan sebagai frontend SPA (Single Page Application) yang mengonsumsi layanan backend melalui REST API. Penggunaan React Router mempermudah pemisahan halaman dan pengaturan alur navigasi. Pendekatan Context API pada autentikasi membuat state user terpusat, sehingga komponen lain dapat mengakses status login tanpa prop drilling. Implementasi interceptor Axios membantu standarisasi request agar token selalu dikirim otomatis saat mengakses endpoint yang diproteksi. Selain itu, adanya kondisi loading, error, dan empty state pada halaman daftar maupun detail post meningkatkan pengalaman pengguna karena aplikasi tetap informatif saat jaringan lambat atau data tidak tersedia.

Kendala yang berpotensi muncul adalah ketidaksesuaian format respons token dari backend, CORS/proxy saat pengembangan lokal, serta endpoint yang tidak tersedia. Pada implementasi ini, kendala format token diantisipasi dengan beberapa opsi pembacaan field respons pada AuthContext dan fallback URL pada service API.

7. Kesimpulan

- Praktikum Modul 12 berhasil menghasilkan aplikasi React Blog Post yang terintegrasi dengan REST API.
- Fitur utama yang berhasil dibuat meliputi autentikasi login, proteksi route, daftar post, dan detail post.
- Penggunaan React Router, Context API, Axios, dan Tailwind CSS membuat aplikasi lebih terstruktur dan mudah dikembangkan.
- Build produksi berhasil dibuat sehingga aplikasi siap dipublikasikan.

8. Lampiran (Link GitHub)

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/DavinAileenskieSofiandy-707012500135>